SwarRakshak Database Design

1. Database objective

SwarRakshak needs to store the minimum metadata required to understand a voice-risk analysis. The database should help the team:

- Create and identify an analysis event.
- Store the call context used for scoring.
- Store the final risk score, level, confidence, and recommendation.
- Store the reasons that contributed to the score.
- Store segment-level audio-analysis results for the timeline.
- Record model and rules versions for reproducibility.
- Record privacy status, including whether temporary audio was deleted.

The database should **not store raw audio bytes for the hackathon MVP**. Audio should remain in temporary private storage only while it is being analyzed and should be deleted afterward by default.

2. Recommended database technology

Use **SQLite** for the hackathon.

SQLite is suitable because it:

- Requires no separate database server.
- Is easy to set up locally.
- Works well with FastAPI and Python.
- Is simple for beginners to inspect and back up.
- Can later be migrated to PostgreSQL if SwarRakshak becomes a multi-user production service.

The design below uses standard SQL that is compatible with SQLite with only minor changes for other relational databases.

3. Design principles

Keep the schema small

The MVP should use a few focused tables rather than a complicated enterprise data model.

Store metadata, not recordings

Audio is sensitive personal information. Store analysis results and reason codes, not the original recording.

Use stable identifiers

Use text IDs such as `evt_...` for analysis events and `usr_...` for users. These are easy to pass through APIs and logs.

Separate current values from repeated details

The overall result belongs in the analysis event table. Individual reasons and audio segments belong in child tables because one event may have many reasons and many segments.

Preserve reproducibility

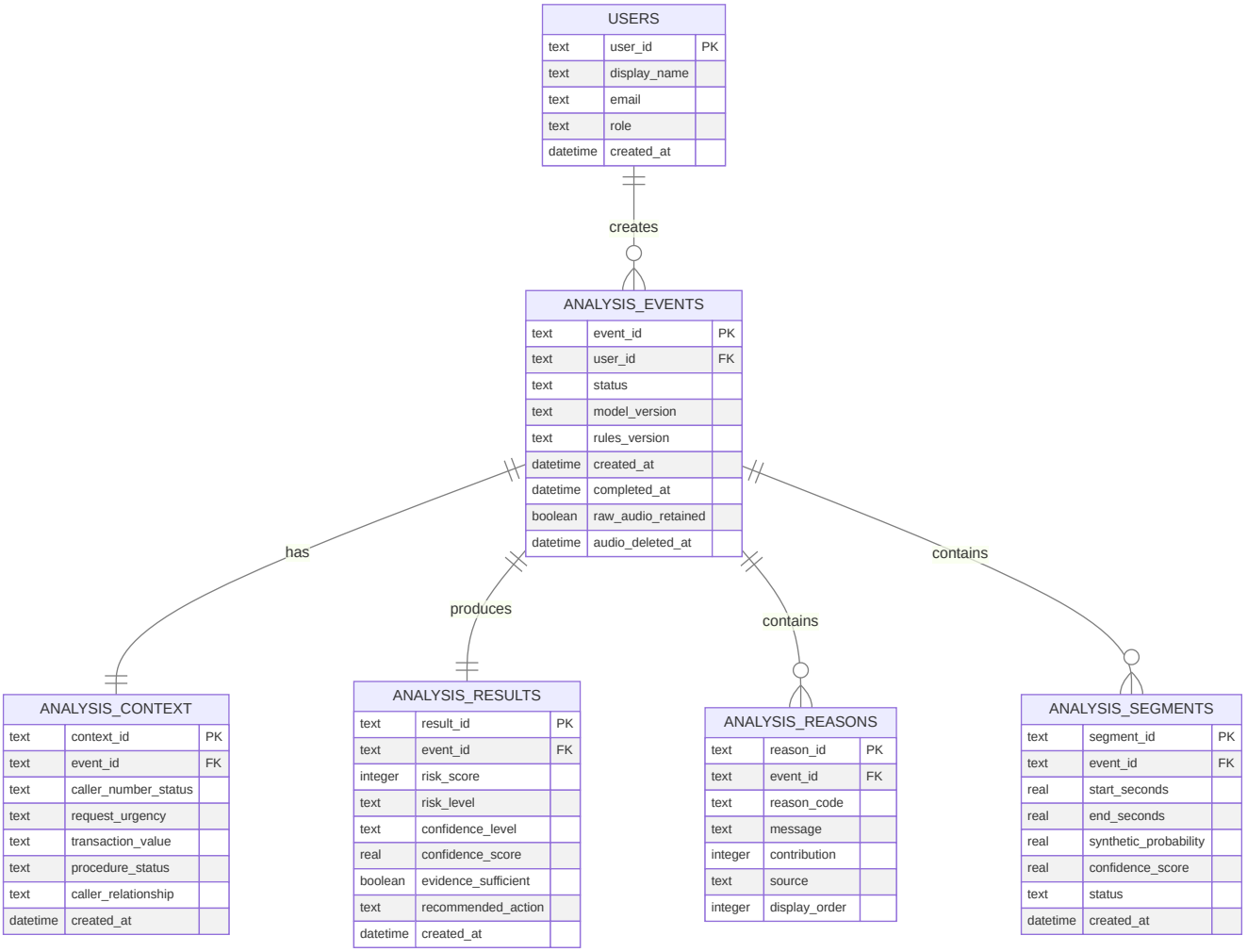
Store the model version and scoring-rules version used for each result. This helps the team understand why two analyses may produce different results after a model or rules update.

4. Entity relationship overview

The MVP can use six tables:

1. **users** — optional users of the prototype.
2. **analysis_events** — one record for each analyzed call sample.
3. **analysis_context** — structured information about the call and requested action.
4. **analysis_results** — final score, risk level, confidence, and recommendation.
5. **analysis_reasons** — reasons and score contributions attached to an analysis.
6. **analysis_segments** — segment-level audio-analysis results used for the timeline.

Relationship diagram



5. Tables and columns

5.1 `users`

Why this table exists

The hackathon can run without full authentication, but a small users table allows the system to associate an analysis with the finance employee who started it. If authentication is not implemented, `user_id` can remain empty.

Columns

Column	Data type	Required?	Key	Description
`user_id`	TEXT	Yes	Primary key	Unique user identifier
`display_name`	TEXT	Yes	—	Name shown in the interface
`email`	TEXT	No	Unique index	Optional contact or login email
`role`	TEXT	Yes	—	Example: `finance_user`, `fraud_analyst`, `admin`
`created_at`	DATETIME	Yes	—	When the user record was created
`is_active`	INTEGER	Yes	—	`1` for active, `0` for disabled

Required fields

- `user_id`
- `display_name`
- `role`
- `created_at`
- `is_active`

Optional fields

- `email`

For a very small demo, the team can insert one demo user and avoid building a login screen.

5.2 `analysis_events`

Why this table exists

This is the central table. It represents one request to analyze one current-call sample. Other tables connect to it through `event_id`.

Columns

Column	Data type	Required?	Key	Description
`event_id`	TEXT	Yes	Primary key	Unique analysis event ID
`user_id`	TEXT	No	Foreign key	User who started the analysis
`status`	TEXT	Yes	—	`started`, `processing`, `completed`, or `failed`
`model_version`	TEXT	Yes	—	Voice/speaker model version used
`rules_version`	TEXT	Yes	—	Risk-scoring rules version
`created_at`	DATETIME	Yes	—	When analysis started
`completed_at`	DATETIME	No	—	When analysis completed
`raw_audio_retained`	INTEGER	Yes	—	`0` by default; `1` only if explicitly retained
`audio_deleted_at`	DATETIME	No	—	When temporary audio was deleted
`failure_code`	TEXT	No	—	Safe machine-readable failure reason

Required fields

- `event_id`
- `status`
- `model_version`
- `rules_version`
- `created_at`

- `raw_audio_retained`

Optional fields

- `user_id`
- `completed_at`
- `audio_deleted_at`
- `failure_code`

Important design decision

Do not include a `current_audio_blob` or `reference_audio_blob` column. The database stores no raw audio in the MVP.

5.3 `analysis_context`

Why this table exists

Voice authenticity is only one part of the decision. This table stores the structured context that can increase risk, such as an unknown number, urgent request, or high-value transfer.

Keeping context in its own table makes the design easy to extend without adding many columns to the central event table.

Columns

Column	Data type	Required?	Key	Description
`context_id`	TEXT	Yes	Primary key	Unique context record
`event_id`	TEXT	Yes	Foreign key, unique	Related analysis event
`caller_number_status`	TEXT	Yes	—	`known`, `unknown`, or `hidden`
`request_urgency`	TEXT	Yes	—	`normal` or `urgent`
`transaction_value`	TEXT	Yes	—	`low`, `medium`, or `high`
`procedure_status`	TEXT	Yes	—	`normal` or `bypass_requested`
`caller_relationship`	TEXT	Yes	—	`customer`, `vendor`, `executive`, or `unknown`
`notes`	TEXT	No	—	Optional short note; do not store confidential call transcripts
`created_at`	DATETIME	Yes	—	When context was recorded

Required fields

- `context_id`
- `event_id`
- `caller_number_status`
- `request_urgency`
- `transaction_value`

- `procedure_status`
- `caller_relationship`
- `created_at`

Optional fields

- `notes`

Privacy guidance

Do not use `notes` to store passwords, account numbers, personal identification numbers, or a full conversation transcript. For the MVP, `notes` can be removed entirely if the team does not need it.

5.4 `analysis_results`

Why this table exists

This table stores the final result displayed to the user. It is kept separate from the event lifecycle because an event may be processing or may fail before a result exists.

There should be at most one result for each completed event in the MVP.

Columns

Column	Data type	Required?	Key	Description
`result_id`	TEXT	Yes	Primary key	Unique result identifier
`event_id`	TEXT	Yes	Foreign key, unique	Related analysis event
`risk_score`	INTEGER	Yes	—	Integer from 0 to 100
`risk_level`	TEXT	Yes	—	`low`, `medium`, `high`, or `inconclusive`
`confidence_level`	TEXT	Yes	—	`low`, `medium`, or `high`
`confidence_score`	REAL	No	—	Optional value from 0.0 to 1.0
`evidence_sufficient`	INTEGER	Yes	—	`1` if evidence was sufficient, otherwise `0`
`recommended_action`	TEXT	Yes	—	Safe next step shown to the user
`created_at`	DATETIME	Yes	—	When the result was generated

Required fields

- `result_id`
- `event_id`
- `risk_score`
- `risk_level`
- `confidence_level`
- `evidence_sufficient`

- `recommended_action`
- `created_at`

Optional fields

- `confidence_score`

Validation rules

- `risk_score` must be between 0 and 100.
- `confidence_score`, when present, must be between 0.0 and 1.0.
- `risk_level` must be one of `low`, `medium`, `high`, or `inconclusive`.
- `evidence_sufficient` must be `0` or `1`.

5.5 `analysis_reasons`

Why this table exists

One analysis can have several reasons. For example, a high-risk result may be caused by synthetic-voice evidence, speaker mismatch, an unknown number, and an urgent high-value request.

A separate table allows the frontend to display a list of reasons and allows the team to show each reason’s contribution to the score.

Columns

Column	Data type	Required?	Key	Description
`reason_id`	TEXT	Yes	Primary key	Unique reason record
`event_id`	TEXT	Yes	Foreign key	Related analysis event
`reason_code`	TEXT	Yes	—	Stable code used by the application
`message`	TEXT	Yes	—	Human-readable explanation
`contribution`	INTEGER	Yes	—	Points added to risk score; may be zero
`source`	TEXT	Yes	—	`audio`, `speaker`, `context`, or `system`
`display_order`	INTEGER	Yes	—	Order shown in the UI

Required fields

All columns are required for the MVP.

Example reason codes

Reason code	Source	Example contribution	User-facing message
`SYNTHETIC_AUDIO_PATTERN`	`audio`	40	Audio contains patterns associated with synthetic speech.
`REFERENCE_VOICE_MISMATCH`	`speaker`	30	Current voice differs from the reference sample.
`UNKNOWN_CALLER_NUMBER`	`context`	10	Caller number is not recognized as trusted.
`HIGH_VALUE_REQUEST`	`context`	10	Request involves a high-value transaction.
`URGENT_OR_BYPASS_REQUEST`	`context`	10	Caller is creating urgency or asking to bypass procedure.
`INSUFFICIENT_AUDIO_EVIDENCE`	`system`	0	Audio quality is not sufficient for a reliable analysis.

5.6 `analysis_segments`

Why this table exists

The voice-analysis module works on short audio windows. This table stores the result for each window so the frontend can show a simple segment timeline.

Columns

Column	Data type	Required?	Key	Description
`segment_id`	TEXT	Yes	Primary key	Unique segment identifier
`event_id`	TEXT	Yes	Foreign key	Related analysis event
`segment_number`	INTEGER	Yes	—	Sequence number within the event
`start_seconds`	REAL	Yes	—	Segment start time
`end_seconds`	REAL	Yes	—	Segment end time
`synthetic_probability`	REAL	No	—	Value from 0.0 to 1.0
`confidence_score`	REAL	No	—	Value from 0.0 to 1.0
`status`	TEXT	Yes	—	`analyzed`, `inconclusive`, or `failed`
`created_at`	DATETIME	Yes	—	When segment result was stored

Required fields

- `segment_id`
- `event_id`
- `segment_number`
- `start_seconds`
- `end_seconds`
- `status`
- `created_at`

Optional fields

- `synthetic_probability`
- `confidence_score`

Validation rules

- `end_seconds` must be greater than `start_seconds`.
- Probability and confidence values must be between 0.0 and 1.0 when present.
- `segment_number` should be unique within an event.

6. Relationships

User to analysis events

Relationship: One user can create many analysis events.

```
users 1 ——— many analysis_events
```

The `analysis_events.user_id` column references `users.user_id`. The field is optional so the MVP can work without authentication.

Analysis event to context

Relationship: One analysis event has exactly one context record in a completed analysis.

```
analysis_events 1 ——— 1 analysis_context
```

The `analysis_context.event_id` field is unique so that an event cannot accidentally receive two context records.

Analysis event to result

Relationship: One analysis event has zero or one result.

```
analysis_events 1 ——— 0..1 analysis_results
```

An event has no result while it is still processing or if it fails.

Analysis event to reasons

Relationship: One analysis event can have many reasons.

```
analysis_events 1 ——— many analysis_reasons
```

Analysis event to segments

Relationship: One analysis event can have many audio segments.

```
analysis_events 1 ——— many analysis_segments
```

7. Required versus optional data summary

Data area	Required for MVP?	Notes
Analysis event	Yes	Central record for every attempt
User account	No	One demo user can be inserted manually
Call context	Yes for completed analyses	Needed for context-aware scoring
Final result	Yes for successful analyses	Stores score and recommendation
Reasons	Yes for useful explanations	At least two for flagged demo cases
Segments	Recommended	Supports the near-real-time timeline
Raw audio	No and discouraged	Keep only in temporary storage
Reference voice profile	No	Use a temporary uploaded sample if needed
Full transcript	No	Avoid storing confidential conversation content

8. Indexes

Indexes improve lookup speed. The hackathon dataset will be small, so these are simple and practical rather than highly optimized.

Recommended indexes

1. `analysis_events.created_at` — list recent analyses.
2. `analysis_events.status` — find processing or failed events.
3. `analysis_events.user_id` — list analyses created by a user.
4. `analysis_results.risk_level` — filter High or Medium risk events.
5. `analysis_reasons.event_id` — load reasons for a result.
6. `analysis_reasons.reason_code` — find events with a particular reason.

7. **`analysis_segments.event_id, segment_number`** — load segments in timeline order.

The unique constraints on `analysis_context.event_id` and `analysis_results.event_id` automatically create useful unique indexes in SQLite.

9. SQL schema

The following schema can be saved as `schema.sql` and executed against SQLite.

```sql
PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS users (
    user_id TEXT PRIMARY KEY,
    display_name TEXT NOT NULL,
    email TEXT UNIQUE,
    role TEXT NOT NULL CHECK (role IN ('finance_user', 'fraud_analyst', 'ac
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    is_active INTEGER NOT NULL DEFAULT 1 CHECK (is_active IN (0, 1))
);

CREATE TABLE IF NOT EXISTS analysis_events (
    event_id TEXT PRIMARY KEY,
    user_id TEXT,
    status TEXT NOT NULL CHECK (
        status IN ('started', 'processing', 'completed', 'failed')
    ),
    model_version TEXT NOT NULL,
    rules_version TEXT NOT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    completed_at DATETIME,
    raw_audio_retained INTEGER NOT NULL DEFAULT 0 CHECK (
        raw_audio_retained IN (0, 1)
    ),
    audio_deleted_at DATETIME,
    failure_code TEXT,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
        ON UPDATE CASCADE
        ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS analysis_context (
    context_id TEXT PRIMARY KEY,
    event_id TEXT NOT NULL UNIQUE,
    caller_number_status TEXT NOT NULL CHECK (
        caller_number_status IN ('known', 'unknown', 'hidden')
    ),
    request_urgency TEXT NOT NULL CHECK (
        request_urgency IN ('normal', 'urgent')
    ),
    transaction_value TEXT NOT NULL CHECK (
        transaction_value IN ('low', 'medium', 'high')
    ),
    procedure_status TEXT NOT NULL CHECK (
```

```
        procedure_status IN ('normal', 'bypass_requested')
    ),
    caller_relationship TEXT NOT NULL CHECK (
        caller_relationship IN ('customer', 'vendor', 'executive', 'unknowr
    ),
    notes TEXT,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (event_id) REFERENCES analysis_events(event_id)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS analysis_results (
    result_id TEXT PRIMARY KEY,
    event_id TEXT NOT NULL UNIQUE,
    risk_score INTEGER NOT NULL CHECK (risk_score BETWEEN 0 AND 100),
    risk_level TEXT NOT NULL CHECK (
        risk_level IN ('low', 'medium', 'high', 'inconclusive')
    ),
    confidence_level TEXT NOT NULL CHECK (
        confidence_level IN ('low', 'medium', 'high')
    ),
    confidence_score REAL CHECK (
        confidence_score IS NULL OR confidence_score BETWEEN 0.0 AND 1.0
    ),
    evidence_sufficient INTEGER NOT NULL CHECK (
        evidence_sufficient IN (0, 1)
    ),
    recommended_action TEXT NOT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (event_id) REFERENCES analysis_events(event_id)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS analysis_reasons (
    reason_id TEXT PRIMARY KEY,
    event_id TEXT NOT NULL,
    reason_code TEXT NOT NULL,
    message TEXT NOT NULL,
    contribution INTEGER NOT NULL DEFAULT 0,
    source TEXT NOT NULL CHECK (
        source IN ('audio', 'speaker', 'context', 'system')
    ),
    display_order INTEGER NOT NULL DEFAULT 1,
```

```
    FOREIGN KEY (event_id) REFERENCES analysis_events(event_id)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS analysis_segments (
    segment_id TEXT PRIMARY KEY,
    event_id TEXT NOT NULL,
    segment_number INTEGER NOT NULL,
    start_seconds REAL NOT NULL CHECK (start_seconds >= 0),
    end_seconds REAL NOT NULL CHECK (end_seconds > start_seconds),
    synthetic_probability REAL CHECK (
        synthetic_probability IS NULL OR
        synthetic_probability BETWEEN 0.0 AND 1.0
    ),
    confidence_score REAL CHECK (
        confidence_score IS NULL OR
        confidence_score BETWEEN 0.0 AND 1.0
    ),
    status TEXT NOT NULL CHECK (
        status IN ('analyzed', 'inconclusive', 'failed')
    ),
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (event_id, segment_number),
    FOREIGN KEY (event_id) REFERENCES analysis_events(event_id)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);

CREATE INDEX IF NOT EXISTS idx_users_role
    ON users(role);

CREATE INDEX IF NOT EXISTS idx_events_created_at
    ON analysis_events(created_at DESC);

CREATE INDEX IF NOT EXISTS idx_events_status
    ON analysis_events(status);

CREATE INDEX IF NOT EXISTS idx_events_user_id
    ON analysis_events(user_id);

CREATE INDEX IF NOT EXISTS idx_results_risk_level
    ON analysis_results(risk_level);

CREATE INDEX IF NOT EXISTS idx_reasons_event_id
```

```
    ON analysis_reasons(event_id);

CREATE INDEX IF NOT EXISTS idx_reasons_code
    ON analysis_reasons(reason_code);

CREATE INDEX IF NOT EXISTS idx_segments_event_order
    ON analysis_segments(event_id, segment_number);
```

10. Example records

The following examples represent a high-risk call from an unknown number. The caller claims to be an executive and requests an urgent high-value transfer.

10.1 Example user

```
INSERT INTO users (
    user_id, display_name, email, role
) VALUES (
    'usr_priya_001',
    'Priya Sharma',
    'priya.demo@example.com',
    'finance_user'
);
```

10.2 Example analysis event

```sql
INSERT INTO analysis_events (
    event_id,
    user_id,
    status,
    model_version,
    rules_version,
    raw_audio_retained,
    completed_at,
    audio_deleted_at
) VALUES (
    'evt_01J9SWR123',
    'usr_priya_001',
    'completed',
    'prototype-audio-1',
    'risk-rules-1',
    0,
    '2026-09-16 14:30:10',
    '2026-09-16 14:30:11'
);
```

10.3 Example context

```
INSERT INTO analysis_context (
    context_id,
    event_id,
    caller_number_status,
    request_urgency,
    transaction_value,
    procedure_status,
    caller_relationship
) VALUES (
    'ctx_01J9SWR123',
    'evt_01J9SWR123',
    'unknown',
    'urgent',
    'high',
    'bypass_requested',
    'executive'
);
```

10.4 Example result

```
INSERT INTO analysis_results (
    result_id,
    event_id,
    risk_score,
    risk_level,
    confidence_level,
    confidence_score,
    evidence_sufficient,
    recommended_action
) VALUES (
    'res_01J9SWR123',
    'evt_01J9SWR123',
    82,
    'high',
    'high',
    0.78,
    1,
    'Pause the transfer and call the person using an independently verifiec
);
```

10.5 Example reasons

```
INSERT INTO analysis_reasons (
    reason_id,
    event_id,
    reason_code,
    message,
    contribution,
    source,
    display_order
) VALUES
(
    'rsn_01J9SWR001',
    'evt_01J9SWR123',
    'SYNTHETIC_AUDIO_PATTERN',
    'Audio contains patterns associated with synthetic speech.',
    40,
    'audio',
    1
),
(
    'rsn_01J9SWR002',
    'evt_01J9SWR123',
    'REFERENCE_VOICE_MISMATCH',
    'Current voice differs from the reference sample.',
    30,
    'speaker',
    2
),
(
    'rsn_01J9SWR003',
    'evt_01J9SWR123',
    'UNKNOWN_CALLER_NUMBER',
    'Caller number is not recognized as trusted.',
    10,
    'context',
    3
),
(
    'rsn_01J9SWR004',
    'evt_01J9SWR123',
    'URGENT_OR_BYPASS_REQUEST',
    'Caller is creating urgency or asking to bypass procedure.',
    10,
```

```
    'context',
    4
);
```

10.6 Example segments

```
INSERT INTO analysis_segments (
    segment_id,
    event_id,
    segment_number,
    start_seconds,
    end_seconds,
    synthetic_probability,
    confidence_score,
    status
) VALUES
(
    'seg_01J9SWR001',
    'evt_01J9SWR123',
    1,
    0.0,
    4.0,
    0.81,
    0.72,
    'analyzed'
),
(
    'seg_01J9SWR002',
    'evt_01J9SWR123',
    2,
    4.0,
    8.0,
    0.91,
    0.76,
    'analyzed'
),
(
    'seg_01J9SWR003',
    'evt_01J9SWR123',
    3,
    8.0,
    12.0,
    0.84,
    0.74,
    'analyzed'
);
```

11. Useful queries

Get the complete result for one event

```sql
SELECT
    e.event_id,
    e.created_at,
    e.status,
    r.risk_score,
    r.risk_level,
    r.confidence_level,
    r.confidence_score,
    r.recommended_action
FROM analysis_events e
JOIN analysis_results r ON r.event_id = e.event_id
WHERE e.event_id = 'evt_01J9SWR123';
```

Get reasons for an event

```sql
SELECT
    reason_code,
    message,
    contribution,
    source
FROM analysis_reasons
WHERE event_id = 'evt_01J9SWR123'
ORDER BY display_order;
```

Get the segment timeline

```sql
SELECT
    segment_number,
    start_seconds,
    end_seconds,
    synthetic_probability,
    confidence_score,
    status
FROM analysis_segments
WHERE event_id = 'evt_01J9SWR123'
ORDER BY segment_number;
```

List recent high-risk events

```sql
SELECT
    e.event_id,
    e.created_at,
    r.risk_score,
    r.risk_level,
    r.recommended_action
FROM analysis_events e
JOIN analysis_results r ON r.event_id = e.event_id
WHERE r.risk_level = 'high'
ORDER BY e.created_at DESC
LIMIT 50;
```

Count risk levels

```sql
SELECT
    risk_level,
    COUNT(*) AS event_count
FROM analysis_results
GROUP BY risk_level
ORDER BY event_count DESC;
```

Find events where raw audio was not deleted

```sql
SELECT
    event_id,
    created_at,
    raw_audio_retained,
    audio_deleted_at
FROM analysis_events
WHERE raw_audio_retained = 1
   OR audio_deleted_at IS NULL;
```

This query supports a simple privacy check during the hackathon.

12. Optional simplified schema

If the team has very little development time, it can begin with only four tables:

1. `analysis_events`
2. `analysis_context`
3. `analysis_results`
4. `analysis_reasons`

The `users` and `analysis_segments` tables can be added later. However, the recommended six-table version is still small enough for a beginner team and provides a better demonstration of the timeline and audit concepts.

13. Implementation notes for beginners

Generate IDs in application code

A simple helper can create IDs such as:

```
usr_<random value>
evt_<random value>
ctx_<random value>
res_<random value>
rsn_<random value>
seg_<random value>
```

Use a collision-resistant method such as UUIDs or secure random tokens.

Use transactions

When creating a completed analysis, insert the event, context, result, reasons, and segments in one database transaction. If one insert fails, roll back the complete operation.

Keep database code separate

Create a small repository or database module rather than writing SQL throughout the frontend and analysis code.

Validate twice

Validate values in the API before writing them to the database. Keep SQL constraints as a second line of defense.

Do not expose internal errors

Log detailed errors on the server, but return a simple message to the user. Do not expose database paths, stack traces, or temporary file names.

Use migrations when the schema changes

For the first demo, running `schema.sql` is acceptable. If the team adds fields later, use a migration tool or numbered SQL files so every teammate can reproduce the database.

14. Future extensions

These should not be added during the first hackathon version unless the core system is complete:

- Consent-managed speaker profiles.
- Organization and department tables.
- Configurable policy and scoring tables.

- Alert-delivery records for email, SMS, or in-app notifications.
- Model-run tables for storing multiple model outputs.
- Data-retention policy tables.
- Investigation cases and analyst comments.
- Multi-tenant enterprise organization support.

A future production schema may also separate personally identifiable information from risk data. That separation is not necessary for the small MVP but would improve privacy and access control at scale.

15. Final recommendation

Use the six-table SQLite schema in this document for the hackathon. It is small enough to understand, supports the complete SwarRakshak workflow, avoids storing raw voice recordings, and can later be migrated to PostgreSQL.

The most important table is `analysis_events`, but the most important privacy decision is what the schema **does not** contain: raw audio, passwords, financial credentials, or full call transcripts.